

Especificação Técnica — Dashboard de Projetos

Documento de referência viva para consultas futuras. Cobre funcionalidades, arquitetura e restrições do sistema como estão em **2026-06-20**. Para o histórico de decisões de cada funcionalidade (problema → decisão → trade-offs), ver os specs e planos individuais em `docs/superpowers/specs/` e `docs/superpowers/plans/`.

1. Visão geral

App Streamlit multipágina para acompanhamento de projetos/financeiro da Intelbras. Público: gerentes de negócio, para análise rápida e tomada de decisão. Acesso protegido por login, com dois perfis: **admin** (CRUD completo) e **visualizador** (somente leitura).

Raiz do app: `dashboard_projetos/` (dentro do repositório `dashboard_projetos/`, que é a raiz do git).
Entry point: `app.py`.

2. Arquitetura e persistência

- **Banco de dados:** Postgres gerenciado (Supabase), via SQLAlchemy + psycopg2-binary. **Não usa SQLite** — SQLite foi abandonado porque o armazenamento do Streamlit Community Cloud é efêmero (qualquer arquivo fora do repositório git é perdido a cada redeploy/reboot).
- `utils/db.py`: camada de persistência.
- Conexão cacheada por processo via `st.cache_resource(_engine())`, com `pool_pre_ping=True`.
- `DATABASE_URL` lido de `st.secrets` (produção: Secrets do Streamlit Cloud; local: `.streamlit/secrets.toml`, **nunca commitado** — está no `.gitignore`).
- `init_db()` / `migrar_db()`: schema idempotente. `migrar_db()` faz `ALTER TABLE ADD COLUMN` para colunas novas sem apagar dados — chamado no topo de cada página, seguro de rodar sempre.
- Todas as queries usam `sqlalchemy.text(...)` com parâmetros nomeados (`:nome`), nunca string interpolada — necessário porque `pd.read_sql` com string pura usa `paramstyle` do driver, enquanto `text()` usa o `paramstyle` portátil do SQLAlchemy. Misturar os dois estilos no mesmo arquivo causa bugs sutis.
- **Datas/timestamps:** armazenados como `TEXT` no formato `YYYY-MM-DD` (ou `YYYY-MM-DD HH:MM:SS` para timestamps), nunca `DATE/TIMESTAMP` nativo — decisão deliberada para preservar a lógica de parsing baseada em string já usada em todo o código (`_parse_data_marco`, `strptime(str(val)[:10], "%Y-%m-%d")`).
- **Valores monetários/horas:** `DOUBLE PRECISION` (não `REAL`).

Armadilha conhecida: cache de módulo do Streamlit

O servidor de desenvolvimento local e o redeploy automático do Streamlit Community Cloud **não recarregam de forma confiável módulos Python já importados** (ex: `utils/data_processor.py`). Editar uma função compartilhada e só dar refresh no navegador (ou esperar o auto-redeploy) pode deixar o processo rodando com a versão antiga em memória, causando `ImportError/KeyError` mesmo com o código novo já no disco/commitado. **Fix:** sempre reiniciar o processo por completo — local: matar e reiniciar o `streamlit run`; produção: usar "Reboot app" no painel do Streamlit Cloud, não confiar só no redeploy automático do push.

3. Páginas (navegação em `app.py`)

```
pages = { "■ Dados": [ "Orçamentos" → pages/0_orcamento.py "Upload de Arquivos" →  
pages/1_upload.py ], "■ Análises": [ "Visão Executiva" → pages/4_visao_executiva.py  
(primeiro item) "Dashboard Financeiro" → pages/2_dashboard.py "Andamento dos Projetos"  
→ pages/3_projetos.py ], }
```

- **Orçamentos** (`0_orcamento.py`): cadastro de orçamento previsto, status do projeto, datas do cronograma (marcos), previsões por período manuais; exportação/importação de orçamentos via Excel.
- **Upload de Arquivos** (`1_upload.py`): upload de planilhas de custos e horas (múltiplos arquivos por vez), histórico de importações, exclusão de importação e "zona de perigo" (apagar tudo).
- **Visão Executiva** (`4_visao_executiva.py`): visão de portfólio ordenada por risco combinado (custo + prazo). Ver seção 6.
- **Dashboard Financeiro** (`2_dashboard.py`): KPIs e gráficos de custo, foco em orçamento x realizado.
- **Andamento dos Projetos** (`3_projetos.py`): prazos, status, cronograma. Duas abas: "Resumo Geral" e "Detalhamento" (drill-down por projeto, com seletor de projeto e projeção de burn rate).

4. Upload de dados — formatos (não alterar sem necessidade real)

Os nomes de coluna abaixo são os **headers originais esperados nas planilhas** (mapeados internamente via `MAP_CUSTOS/MAP_HORAS` em `utils/data_processor.py`, com tolerância a acentos/caixa). Mudar esses formatos quebra a experiência de quem já tem planilhas no padrão atual — qualquer melhoria de exibição deve ser feita na camada de apresentação, não no formato de entrada.

Custos (CSV ; ou XLSX): Data, Ano, Mês, Filial, Área, Centro de Custo, Conta, Cód. Parceiro Negócio, Parceiro Negócio, Histórico, Realizado. Colunas obrigatórias após mapeamento: `centro_de_custo`, `realizado`, `mes`, `ano`.

Horas (CSV ; ou XLSX): Período, C.Custo, Descrição Ordem Interna, Centro de Lucro, Descrição C.Lucro, Matrícula, Nome, CC Origem, Descrição CC Origem, Hs Nor, Tipo de Projeto, Cód Produto, Descrição Produto, CATEGORIA, ATIVIDADE, DETALHES, C.Custo - Descrição Ordem Interna, Matrícula - Nome, Segmento. Colunas obrigatórias após mapeamento: `c_custo`, `hs_nor`, `nome`, `periodo`.

- `centro_de_custo/c_custo` é a **chave de cruzamento** entre custos e horas (truncada para os 9 primeiros caracteres).
- Cada arquivo enviado é acumulado no histórico (`importacoes`); reenviar o mesmo nome de arquivo do mesmo tipo é detectado e ignorado (`_ja_importado`).
- Orçamento é importável via Excel exportado anteriormente (abas `Orçamentos` e `Previsoes`) — função `importar_orcamento_de_excel` em `utils/data_processor.py`. Células vazias/NaN são tratadas como nulas via helpers `_texto_celula/` `_numero_celula` (não usar `str(val)` direto, gera a string "nan").

5. Filtros (Dashboard, Andamento, Visão Executiva)

Componente compartilhado em `utils/data_processor.py`:

- `aplicar_filtros(df, df_custos_raw, projetos_sel, anos_sel, meses_sel, status_sel) -> pd.DataFrame` — lógica pura, testável.
- `render_filtros_sidebar(df, df_custos_raw) -> pd.DataFrame` — desenha a sidebar e chama `aplicar_filtros`.

Convenção única para os 4 filtros: seleção vazia = sem filtro (mostra tudo). Projetos e Mês iniciam vazios; Ano e Status mantêm defaults curados no primeiro carregamento (`anos_default = ano corrente`; `status_ativos = exclui "Cancelado"/"Lançado"`). O botão "■ Limpar filtros" zera os 4 ([]).

Restrição de API do Streamlit: o botão de limpar filtros **precisa** usar `on_click=callback`, nunca `if st.button(...): st.session_state[key] = ...` direto no corpo — atribuir a `st.session_state[key]` depois que um widget com aquele `key` já foi instanciado no mesmo run lança `StreamlitAPIException`. Mesma regra vale para qualquer outro "reset de widget" futuro.

6. Visão Executiva — risco de portfólio

`calcular_risco_portfolio(df, df_custos_raw) -> pd.DataFrame` em `utils/data_processor.py`. Para cada projeto, combina dois sinais:

Risco de custo — reaproveita `projecao_burn_rate` (já existente, calcula custo projetado pelo ritmo médio mensal de gasto): - `pct_projetado is None` (sem orçamento cadastrado) → indeterminado (tratado como baixo risco, motivo informativo "Sem orçamento cadastrado") - `> 100%` → estouro (alto) - `>= 80%` → atenção (médio) - `< 80%` → saudável (baixo)

Risco de prazo — checa os 4 marcos com par previsto/realizado (`MARCOS_RISCO`): (`prev_viabilidade, real_viabilidade, "Viabilidade"`), (`prev_qualidade, real_qualidade, "Qualidade"`), (`prev_aprov_lancamento, real_aprov_lancamento, "Aprovação para Lançamento"`), (`prev_lancamento, real_lancamento, "Lançamento"`). `data_inicio` não tem par "real" e **não** é checado para atraso. **Qualquer atraso > 0 dias já conta como risco**, sem tolerância.

Risco combinado: ■ Alto se (estouro de custo OU algum atraso); ■ Médio se (atenção E sem atraso); ■ Baixo nos demais casos.

■■ Esses limiares (80/100) são isolados desta função — não substituem os limiares já existentes e inconsistentes entre si em outras partes do código: `cor_status` usa 90/100, `rotulo_consumo` usa 80/100, a tabela do Dashboard usa 70/90/100. Unificar isso é uma melhoria futura documentada, não feita ainda (decisão deliberada para não alterar comportamento de telas já em uso sem ter sido pedido).

■■ **Armadilha real do pandas:** quando o `DataFrame` resultante tem **múltiplos projetos** misturando `None` com valores reais nas colunas `pct_projetado` (float) ou `proxima_fase` (string), o pandas converte `None` em `NaN` para aquela coluna (inferência de dtype). `NaN` é *truthy* em `bool()` e `NaN is not None` é `True`. Isso já causou um bug real (texto "nan%" exibido, `AttributeError` ao tentar formatar uma data que era `None`). **Sempre usar `pd.isna()/pd.notna()`** ao consumir essas colunas fora da própria função — nunca `is None` ou `truthiness` direta. Esse bug só aparece com 2+ projetos no resultado; testes de uma linha não o expõem.

Página (`4_visao_executiva.py`): contadores Alto/Médio/Baixo; cards grandes só para Alto/Médio (Baixo fica num `st.expander` compacto); botão "■ Ver detalhamento do projeto" em cada card que navega para Andamento dos Projetos já com o projeto e a aba "Detalhamento" selecionados (ver seção 7); exportação CSV e PDF (`gerar_relatorio_risco_pdf`).

7. Navegação programática entre páginas

Padrão usado para "Ver detalhamento" (Visão Executiva → Andamento dos Projetos, aba Detalhamento, projeto pré-selecionado):

```
# Origem: define o estado ANTES de trocar de página
st.session_state["ir_para_projeto"] = r["projeto"] # código do CC
st.session_state["ir_para_tab"] = "■ Detalhamento" # label exata da aba
st.switch_page(str(_ROOT / "pages" / "3_projetos.py")) # path absoluto # Destino: lê e
CONSUME (.pop) o estado, define o valor no widget # correto ANTES dele ser instanciado
nesse run _tab_alvo = st.session_state.pop("ir_para_tab", None) tabs = st.tabs(...),
default=_tab_alvo) # st.tabs aceita default= (Streamlit 1.58+) _cc_alvo =
st.session_state.pop("ir_para_projeto", None) if _cc_alvo: # ... resolve _cc_alvo para
o valor exibido no selectbox (nome_projeto # agrupado, não o CC bruto) e grava em
st.session_state[key do widget] st.session_state["projeto_detalhe_selecionado"] =
nome_resolvido projeto = st.selectbox(..., key="projeto_detalhe_selecionado")
```

Por que index= não funciona aqui: um `st.selectbox` sem `key=` guarda implicitamente a última escolha do usuário; um `index=` recalculado a cada run é **ignorado** depois que o widget já foi interagido uma vez. A forma confiável é sempre: gravar o valor desejado em `st.session_state[key]` **antes** do widget (com `key=` explícito) ser instanciado nesse run — mesmo padrão já usado pelos filtros (seção 5).

`st.tabs(..., default=...)` existe nesta versão do Streamlit (1.58) e permite abrir numa aba específica por padrão — não confundir com versões mais antigas do Streamlit, que não tinham esse parâmetro.

8. Cronograma / marcos do projeto

Tabela `orcamentos_cronograma`, chave primária `projeto` (= Centro de Custo). Colunas de data, todas TEXT formato YYYY-MM-DD: `data_inicio` (sem par "real"), e os 4 pares `prev_viabilidade/real_viabilidade`, `prev_qualidade/real_qualidade`, `prev_aprov_lancamento/real_aprov_lancamento`, `prev_lancamento/real_lancamento`.

Regra de atraso usada em todo o código (`detectar_excecoes`, `calcular_risco_portfolio`, `badges` de `3_projetos.py`): marco atrasado se `real` existe e é depois de `prev` (atraso consumado), OU `real` não existe e hoje é depois de `prev` (atraso em andamento). `detectar_excecoes` (usado nos alertas de Dashboard/Andamento) só checa o marco de **Lançamento** para a categoria atrasados — diferente de `calcular_risco_portfolio`, que checa os 4 marcos. Isso é intencional: correção feita apenas na função nova, não retroaplicada às páginas existentes.

9. Funcionalidade órfã conhecida

`previsoes_periodo` ("Previsão por Período"): tabela e funções de CRUD existem (`salvar_previsao_periodo`, `carregar_previsoes_projeto`, `carregar_todas_previsoes`), e dados podem ser **importados** via a aba "Previsoes" do Excel de orçamento. **Nenhuma página exibe esses dados**. Decisão registrada: deixar como está por ora (não remover, não exibir); revisar essa funcionalidade é uma melhoria futura possível.

10. Autenticação e perfis

`utils/auth.py`: - `exigir_login()` -> None — barreira obrigatória, renderiza a tela de login e chama `st.stop()` se não autenticado. - `perfil_admin()` -> bool — True se `st.session_state["perfil"] == "admin"`. - `logout()` -> None. - Secret AUTH_USERS: usuários separados por ;, cada um no formato `usuario:perfil:hash_sha256_da_senha` (`perfil = "admin" ou "visualizador"`). Ex.: `"admin:admin:5e88...;consulta:visualizador:9f86..."`. - Cada página individualmente decide o que esconder/desabilitar para visualizador chamando `perfil_admin()` — não há um middleware central de autorização por página, é responsabilidade de cada página.

11. Exportação (CSV / PDF)

- CSV: `df.to_csv(index=False).encode("utf-8") + st.download_button`, padrão em todas as páginas com dados tabulares.
- PDF: `utils/pdf_report.py` (reportlab/Platypus), paisagem A4.
- `gerar_relatorio_pdf(df, titulo_pagina, filtros, excecoes, incluir_status)` — usado por Dashboard Financeiro e Andamento dos Projetos. Tabela de "Pontos de Atenção" vem de `detectar_excecoes`.
- `gerar_relatorio_risco_pdf(df_risco, filtros)` — usado pela Visão Executiva. Tabela detalhada só dos projetos em risco Alto/Médio (motivos, % projetado, próxima fase); Baixo risco só conta no resumo, não detalha (mesma lógica visual da página).
- **Sem testes automatizados** para este módulo (precedente estabelecido) — verificação é manual, gerando o PDF na página.

12. Testes (pytest)

- `tests/conftest.py`: fixture de sessão autouse chama `init_db()`/`migrar_db()` uma vez; fixture autouse por teste limpa linhas cujo campo de identificação contém o marcador `__teste_pytest__` (convenção: nomear arquivos/projetos de teste incluindo esse marcador, para limpeza automática).
- **Os testes rodam contra o Supabase Postgres real**, não um mock nem um banco local — precisa de `.streamlit/secrets.toml` com `DATABASE_URL` válido para rodar localmente.
- Cobertura: `utils/db.py` (CRUD completo), `utils/data_processor.py` (filtros, upload, importação de orçamento, risco de portfólio) — todas funções puras ou que tocam o banco real.
- **Não coberto** (precedente deliberado): páginas (`pages/*.py`, UI Streamlit) e `utils/pdf_report.py` — verificação manual no navegador.
- 41 testes no momento da escrita deste documento.

13. Ambiente de desenvolvimento local (Windows)

- Interpretador Python: `C:\Program Files\Python313\python.exe`. O Bash tool deste ambiente não tem `python/python3` confiável no PATH — sempre usar o caminho completo, ou via PowerShell com `$env:PATH = "C:\Users\<user>\AppData\Roaming\Python\Python313\Scripts;C:\Program Files\Python313;$env:PATH"` antes de chamar `pytest/streamlit` (os console scripts ficam em `Scripts`, fora do PATH padrão).
- Redes corporativas/VPN podem bloquear TCP direto para IPs da AWS (pooler do Supabase em 5432/6543, e até 443 em IP bruto da AWS), enquanto liberam HTTPS para serviços atrás de Cloudflare (ex: `*.supabase.co`, `github.com`). Se o ambiente local de repente não conseguir conectar no Postgres, suspeitar da rede antes de suspeitar do código — confirmar com `Test-NetConnection/tracert` antes de investigar "bug".

14. Deploy / produção

- Repositório GitHub: `lucasfinal/dashboard_projetos`, branch `main`. Workflow estabelecido: push direto na `main` (sem PR), exceto quando explicitamente decidido usar uma `worktree/branch` isolada para

uma funcionalidade maior (feito uma vez, para a Visão Executiva).

- Streamlit Community Cloud faz redeploy automático a cada push — mas ver a armadilha de cache de módulo na seção 2; um "Reboot app" manual costuma ser necessário para garantir que o processo novo rode com o código mais recente.
- `.github/workflows/auto-commit.yml` (commit vazio a cada 4h, para manter o app "aquecido"/evitar sleep) foi **removido** — ele disparava deploys que arriscavam perda de dados na época do SQLite efêmero. Hoje seria inofensivo (Postgres persiste), mas continua removido.
- Segredos de produção: painel do Streamlit Cloud → Settings → Secrets (`DATABASE_URL`, `AUTH_USERS`). Nunca commitados.

15. Convenções de código a seguir

- Imports de `datetime` dentro de `utils/data_processor.py` são **locais à função** (`from datetime import datetime as _dt`), não no topo do arquivo — padrão repetido em `projecao_burn_rate`, `detectar_excecoes`, `ano_corrente_str`, `calcular_risco_portfolio`. Seguir esse padrão em funções novas do mesmo arquivo, em vez de promover o import para o topo.
- Toda query SQL usa `text()` do SQLAlchemy com parâmetros nomeados — nunca montar SQL com f-string/%/concatenação.
- Funções de cálculo/lógica pura (sem `st.*`) ficam separadas das funções de renderização (`render_*`) dentro do mesmo módulo, para permitir teste unitário das primeiras sem precisar de um app Streamlit rodando.